

Workshop No. 1

Requirements, User Stories and Story Mapping

Spynet



Watching every part | Auditing every weakness

Grupo 5 — Los Extraditables

Juan Diego Rozo Álvarez	Backend
Santiago Alejandro Rojas Feo	Backend
Alejandro Medina Rojas	Backend
Misael Jesus Flores Anave	DevOps
Cristian David Arcia Quintero	Frontend
Marlon David Pabon Muñoz	CI/CD/Documentación

1. Contexto del Sistema y Requerimientos

Spynet es una aplicación web diseñada para analizar y mapear la composición tecnológica de páginas web. La aplicación permitirá a los usuarios ingresar una URL y obtener un análisis estructurado de las tecnologías utilizadas, junto con información complementaria sobre el dominio y su infraestructura.

La motivación de este proyecto surge de la necesidad de comprender el ecosistema tecnológico detrás de las aplicaciones web modernas. Desarrolladores, estudiantes e investigadores requieren herramientas que les permitan identificar cómo están contruidos los sitios web, comparar diferentes stacks tecnológicos y detectar patrones o tendencias. Aunque existen herramientas similares, muchas son propietarias o limitadas en funcionalidad. Spynet busca ofrecer una solución simplificada pero extensible, que no solo analice sitios web, sino que también almacena los resultados en una base de datos estructurada, permitiendo análisis comparativos y reutilización de la información a través de una API REST.

1.1 Contexto del Sistema (Naturaleza y Sector)

Naturaleza del sistema:

Spynet es una aplicación web diseñada para analizar y mapear la composición tecnológica de páginas web. La aplicación permitirá a los usuarios ingresar una URL y obtener un análisis estructurado de las tecnologías utilizadas, junto con información complementaria sobre el dominio y su infraestructura.

Sector del sistema:

El sector de Spynet es variado, sin embargo se centra en el sector tecnológico y la ciberseguridad. Su propósito es dar visibilidad sobre la infraestructura y las tecnologías que componen a ciertos sitios web, colaborando a estos sectores de forma práctica, más específicamente a equipos de desarrollo que busquen inspirarse en sitios existentes para definir su propio stack tecnológico e investigadores y profesionales en el área de la seguridad que necesiten comprender e identificar tecnologías desactualizada o vulnerables e identificar la infraestructura pública de un dominio

1.2 Actores

Por la naturaleza y funcionalidad de Spynet, no es necesario tener más de dos actores principales, el usuario y el sistema. El usuario es cualquier persona que usa la aplicación mediante la interfaz web

para analizar las URL que quiera, esto para analizar, consultar y explorar estadísticas de las webs; dentro de estos actores podemos encontrar ciertos patrones demográficos, así que estos datos de la población no serán de utilidad para el desarrollo de una herramienta cómoda y sencilla, entre estas características está:

- Rango de edad: 16-45, Aunque este rango es subjetivo, suponemos que las personas que más usarán la herramienta estarán de cierta manera, dentro del campo de tecnologías relacionadas a la ciberseguridad, siendo un campo ocupado principalmente por gente joven
- Género: Por el contexto actual del sector tecnológico, la población será predominantemente masculina, sin embargo la herramienta es completamente neutra, así que puede ser usada por cualquier persona
- Nivel educativo: Técnico/Tecnólogo/Universitario, se prevé que la herramienta sea usada principalmente por personas con estudios en carreras directamente relacionadas con tecnologías de la información
- Intereses: Seguridad informática, hacking ético, Desarrollo web, arquitectura de software, etc.
- Experiencia técnica: Se espera que el usuario tenga conocimientos previos en conceptos como DNS, HTTP, dominos web, stack tecnológicos, etc.
- Dispositivos: Principalmente computador

El sistema es un actor interno que maneja los servicios de análisis, detección de tecnologías, Wayback Machine, y en general todas las herramientas que se implementen en el proyecto. Este no tendrá intervención humana directa, así que no se le puede dar características demográficas.

1.3 Entidades del Dominio

Análisis:

Representa el resultado completo de procesar la información que tenga la web con la URL que coloca el usuario, toda esta información recolectada se agrupa, y cada vez que un usuario analiza un sitio web, se genera un nuevo análisis que de nuevo, almacena los datos obtenidos. Las características son las siguientes:

- URL: dirección del sitio web que fue analizado
- Estado: Si hubo algún error o no
- Fecha de creación: Momento en el que se hizo el análisis

Tecnología:

Representa cada tecnología identificada dentro de un sitio web analizado, como frameworks de frontend, tecnologías de backend o proveedores de CDN. Una misma tecnología puede aparecer en múltiples análisis. Las características son las siguientes:

- Nombre: denominación de la tecnología detectada (ej: React, Django, Cloudflare)-
- Categoría: clasificación según su rol en el sitio (Frontend, Backend)
- Nivel de confianza: qué tan seguro está el sistema de que esa tecnología realmente está siendo usada
- Evidencia: patrón, header o señal que originó la detección

Registro DNS:

Contiene la información del sistema de nombres de dominio asociada al sitio analizado, permitiendo entender cómo está configurada la infraestructura de red y qué proveedores están involucrados. Las características son las siguientes:

- Dirección IP: dirección numérica a la que apunta el dominio
- Servidores de nombre: servidores encargados de gestionar el dominio
- Registros de correo: servidores de correo electrónico asociados al dominio

Información WHOIS:

Almacena los datos públicos de registro del dominio analizado, permitiendo conocer quién lo registró, cuándo fue creado y cuándo vence su registro. Las características son las siguientes:

- Registrador: empresa o entidad que registró el dominio
- Fecha de creación: cuándo fue registrado el dominio por primera vez
- Fecha de expiración: cuándo vence el registro actual del dominio
- Antigüedad: años que lleva activo el dominio desde su creación
-

Geolocalización:

Representa la ubicación física del servidor donde está alojado el sitio web, obtenida a partir de la dirección IP del dominio. Permite conocer en qué parte del mundo reside físicamente la infraestructura del sitio. Las características son las siguientes:

- País: país dónde está ubicado el servidor
- Ciudad: ciudad específica donde se encuentra el servidor
- ISP: proveedor de internet que administra la conexión del servidor
- Organización: entidad responsable de la dirección IP
- Coordenadas: latitud y longitud para representar la ubicación en un mapa interactivo

Snapshot:

Representa una captura histórica del sitio web almacenada en Wayback Machine, permitiendo rastrear cómo ha evolucionado el sitio a lo largo del tiempo tanto en diseño como en composición tecnológica. Las características son las siguientes:

- Fecha de captura: momento en que el sitio fue archivado
- URL de acceso: enlace directo para visualizar esa versión archivada en archive.org
- Total de capturas: número acumulado de veces que el sitio ha sido archivado históricamente

1.4 Reglas de negocio

RN-01 — Formato y validación de URL:

Toda URL ingresada por el usuario debe ser procesada por el sistema. Si la URL no incluye protocolo (ej: github.com), el sistema debe agregar automáticamente https:// antes de procesarla. No se debe retornar un error por este motivo.

RN-02 — Análisis pasivo únicamente:

El sistema solo puede interactuar con los sitios web mediante peticiones HTTP GET. Está estrictamente prohibido ejecutar, inyectar o interactuar activamente con el código del sitio analizado. No se almacenan credenciales ni datos sensibles de ningún tipo.

RN-03 — Tolerancia a fallos de servicios externos:

Si alguno de los servicios externos (DNS, WHOIS, Geolocalización, Wayback Machine, IA) falla durante un análisis, el sistema debe retornar null en ese campo específico y continuar con el resto del análisis sin interrumpirse. Un fallo parcial no debe considerarse un fallo total.

RN-04 — Umbral de confianza para detección de tecnologías:

Una tecnología solo puede ser reportada como detectar si su nivel de confianza supera un umbral mínimo basado en al menos dos señales fuertes. Detectar un único patrón débil no es suficiente para confirmar el uso de una tecnología, con el fin de evitar falsos positivos.

RN-05 — Tiempo máximo de análisis:

El análisis completo de un sitio (tecnologías + DNS + WHOIS + GeoIP + Wayback) debe completarse en un tiempo máximo de 20 segundos bajo condiciones normales de red. Si el sitio no responde o se produce un timeout, el sistema debe retornar un error descriptivo con código 502.

RN-06 — Análisis retroactivo de snapshots:

Cuando se analiza un snapshot histórico de Wayback Machine, el sistema únicamente puede ejecutar análisis pasivo del stack tecnológico (frontend/backend) sobre el HTML crudo. Quedan explícitamente excluidas las consultas sincrónicas de red como DNS, WHOIS, geolocalización o recursión de nuevos snapshots.

RN-07 — Historial de análisis sin duplicados activos:

Cada solicitud de análisis genera un nuevo registro independiente en la base de datos, incluso si la URL ya fue analizada anteriormente. El sistema no bloquea análisis repetidos, pero permite al usuario consultar el historial para reutilizar resultados previos.

RN-08 — Privacidad WHOIS:

Si un dominio tiene activada la privacidad WHOIS, el sistema debe retornar los campos disponibles públicamente y marcar como null los que estén ocultos, sin interrumpir el análisis ni generar un error.

RN-09 — Estadísticas globales:

El ranking de tecnologías más detectadas debe incluir únicamente las 20 tecnologías más frecuentes, organizadas por categoría (Frontend, Backend, CDN). Este ranking se calcula sobre el total acumulado de todos los análisis almacenados.

1.5 Requisitos Funcionales

Los requisitos funcionales describen las capacidades que el sistema debe ofrecer a sus actores: el **Usuario** (quien ingresa URLs y consulta resultados) y el **Sistema** (que procesa solicitudes y gestiona datos).

ID	Nombre	Descripción
----	--------	-------------

RF-01	Analisis de sitio web	El sistema debe aceptar una URL como entrada y ejecutar un análisis automático que detecte las tecnologías usadas por el sitio (frontend, backend, servidores, CDN).
RF-02	Deteccion de tecnologías frontend	El sistema debe identificar frameworks de frontend como React, Vue, Angular, Next.js y Nuxt.js, mediante analisis del HTML, etiquetas script y headers HTTP.
RF-03	Detección de tecnologías backend	El sistema debe identificar tecnologías de backend como Django, Node.js, Laravel, WordPress y PHP, a través de headers HTTP, cookies y patrones en el HTML.
RF-04	Detección de CDN y hosting	El sistema debe identificar el proveedor de CDN u hosting del sitio (Cloudflare, AWS, Vercel, Netlify) usando nameservers DNS, headers y rangos de IP.
RF-05	Consulta de registros DNS	El sistema debe consultar los registros DNS del dominio (A, AAAA, MX, NS, TXT) usando dnspython y almacenar la IP resuelta, nameservers y registros de correo.
RF-06	Consulta WHOIS del dominio	El sistema debe obtener datos WHOIS del dominio analizado: registrador, fecha de creación, fecha de expiración y calcular la antigüedad del dominio en años.
RF-07	Geolocalizacion de IP	El sistema debe localizar la IP del dominio consultando una API pública (ip-api.com) y retornar al país, ciudad, ISP y organización que administra esa dirección IP.
RF-08	Historial con Wayback Machine	El sistema debe consultar la API CDX de archive.org para obtener snapshots históricos del sitio: cantidad total de archivos y los últimos timestamps con URL de acceso.
RF-09	Almacenamiento de resultados	El sistema debe persistir cada análisis en base de datos relacional, incluyendo tecnologías detectadas, datos DNS, WHOIS, geolocalización y snapshots de Wayback.

RF-10	Consulta de historial de análisis	El sistema debe exponer un endpoint GET que liste todos los análisis realizados ordenados cronológicamente, con su URL, estado y cantidad de tecnologías detectadas.
RF-11	Detalle de analisis	El sistema debe permitir consultar el detalle completo de un análisis específico por su ID, incluyendo todas las tecnologías con su evidencia de detección.
RF-12	Comparación de sitios	El sistema debe comparar dos análisis e identificar tecnologías en común, exclusivas del sitio A y exclusivas del sitio B.
RF-13	Estadísticas globales	El sistema debe exponer estadísticas: total de análisis, total de tecnologías detectadas y ranking de las 20 tecnologías más frecuentes con su categoría.
RF-14	Manejo de errores	El sistema debe manejar fallos graciosamente: si un servicio externo (WHOIS, DNS, Wayback, GeoIP) falla, retorna null en ese campo sin interrumpir el resto del análisis.

*RF-15	Interfaz web de análisis	El sistema debe proveer una interfaz web gráfica (UI) adaptada para que el usuario ingrese URLs de forma intuitiva y visualice los resultados consolidados del análisis estructurados por módulos categóricos.
*RF-16	Renderizado de geolocalización en mapa	El sistema debe tomar las coordenadas geográficas (latitud y longitud) provistas por el GeoService y proyectar la ubicación del servidor de alojamiento en un componente de mapa interactivo con un marcador de posición dinámico.
*RF-17	Línea de tiempo y miniaturas de historial	El sistema debe renderizar de forma cronológica una línea de tiempo visual con las miniaturas de los snapshots históricos obtenidos mediante la API CDX de archive.org.
*RF-18	Graficación de analíticas globales	El sistema debe procesar el ranking de las 20 tecnologías más frecuentes provisto por el endpoint de estadísticas globales y proyectarlo visualmente mediante componentes de gráficas interactivas.
*RF-19	Análisis retroactivo de snapshots históricos	El sistema debe ser capaz de procesar el código HTML crudo de una URL de snapshot histórico proveída por la Wayback Machine para ejecutar un análisis pasivo exclusivamente de su stack tecnológico (frontend/backend). Por definición, se excluyen de este análisis derivado las consultas de red sincrónicas (DNS, WHOIS, geolocalización o recursión de snapshots).
*RF-20	Análisis interpretativo asistido por IA	El sistema debe consumir una API de Inteligencia Artificial externa y gratuita para procesar de forma holística los datos crudos obtenidos del análisis (DNS, WHOIS, Tecnologías, etc.) y generar un reporte interpretativo en lenguaje natural sobre el estado del sitio web.

1.6 Requisitos No Funcionales

Los requisitos no funcionales definen los atributos de calidad del sistema, relacionados con rendimiento, seguridad, mantenibilidad y portabilidad.

ID	Nombre	Descripción
RNF-01	Rendimiento	El análisis completo de un sitio (tecnologías + DNS + WHOIS + GeoIP + Wayback) debe completarse en menos de 20 segundos bajo condiciones normales de red.
RNF-02	Disponibilidad	La API debe tener disponibilidad del 99% en el entorno de producción. Los fallos en servicios externos no deben derribar el sistema.
RNF-03	Escalabilidad	La arquitectura modular basada en patrones de diseño (Factory, Singleton, Facade) debe permitir agregar nuevos detectores o servicios sin modificar la lógica existente.
RNF-04	Seguridad	El sistema no debe ejecutar el código de los sitios analizados. Las URLs son consultadas de forma pasiva (HTTP GET). No se almacenan credenciales ni datos sensibles.

RNF-05	Mantenibilidad	El código debe seguir principios OOP, tener cobertura de tests automatizados superior al 70% y documentación clara de arquitectura y patrones usados.
RNF-06	Usabilidad de la API	La API REST debe seguir convenciones estándar: verbos HTTP correctos, códigos de estado apropiados (200, 201, 400, 404, 502) y respuestas JSON consistentes.
RNF-07	Portabilidad	El sistema debe poder ejecutarse en cualquier entorno con Python 3.10+ mediante Docker, garantizando reproducibilidad entre desarrollo y producción.

RNF-08	Trazabilidad	Cada tecnología detectada debe incluir el campo evidence que explica qué patrón o header originó la detección, facilitando la verificación de resultados.
---------------	---------------------	---

2. User Stories

Las historias se redactan para los dos actores definidos en la propuesta del proyecto: **Usuario** (desarrollador, estudiante o investigador que analiza sitios web) y **Sistema** (actor interno responsable de procesar las solicitudes). El formato usado es: *Como [rol], quiero [acción] para que [beneficio]*, con criterios de aceptación en formato Given / When / Then.

US-01	Analizar un sitio web	Usuario
Como usuario , quiero ingresar una URL y obtener el perfil tecnológico completo del sitio, para conocer qué frameworks, servidores y servicios utiliza.		
Given	el usuario envía POST /api/analyze/ con {"url": "https://ejemplo.com"}	
When	el sistema procesa la solicitud correctamente	
Then	retorna JSON con tecnologías detectadas (nombre, categororía, confianza, evidencia), datos DNS, WHOIS, geolocalización y snapshots de Wayback, con status 201.	
Given	el usuario envía una URL sin protocolo como "github.com"	
When	el sistema recibe la solicitud	
Then	agrega automáticamente "https://" y procesa la URL sin retornar error.	
Given	el sitio no responde o se produce un timeout	
When	el sistema agota el tiempo de espera	
Then	retorna status 502 con mensaje descriptivo del error.	

US-02	Consultar registros DNS del dominio	Usuario
Como usuario , quiero ver los registros DNS del sitio analizado, para entender su infraestructura: IP, servidores y proveedores de correo.		
Given	el sistema resuelve el dominio de la URL ingresada	
When	el DNS Service consulta los registros usando dnspython	
Then	La respuesta incluye la IP (registro A), los nameservers (NS) y los registros de correo (MX) del dominio.	
Given	los nameservers contienen el patron "cloudflare.com"	
When	el sistema procesa los registros NS	
Then	El CDN Detector usa esa información para confirmar el uso de Cloudflare con un nivel de confianza de al menos 85%.	

US-03	Consultar datos WHOIS del dominio	Usuario
Como usuario , quiero ver los datos WHOIS del sitio analizado, para conocer quien lo registró, cuando fue creado y cuando expira.		
Given	el sistema completa el análisis de una URL válida	
When	el Whois Service consulta los datos del dominio	
Then	La respuesta incluye registrar, created date, expiration date y domain age years calculado automáticamente.	
Given	el dominio tiene privacidad activada (WHOIS privacy)	
When	el sistema intenta consultar los datos	
Then	retorna los campos disponibles y marca como null los ocultos, sin interrumpir el resto del análisis.	

US-04	Ver geolocalización de la IP del sitio	Usuario
Como usuario , quiero saber en qué país y con qué proveedor está alojado el sitio analizado, para entender dónde está físicamente su infraestructura.		
Given	el sistema resuelve la IP del dominio	
When	consulta la API publica de ip-api.com con esa IP	
Then	La respuesta incluye país, ciudad, ISP y organización asociados a esa IP.	
Given	la API de geolocalización no está disponible	
When	se produce un error de conexion o timeout	
Then	el sistema retorna null en el campo geo y continua con el resto del análisis sin interrupciones.	

US-05	Ver historial de snapshots con Wayback Machine	Usuario
Como usuario , quiero ver cuantas veces ha sido archivado el sitio en Wayback Machine y en qué fechas, para entender su historial y evolución en el tiempo.		
Given	el usuario analiza cualquier URL	
When	el WaybackService consulta la CDX API de archive.org	
Then	La respuesta incluye snapshot_count (total de archivados) y los últimos 5 timestamps con su URL de acceso a la versión archivada.	
Given	el sitio no tiene historial en Wayback Machine	
When	la API devuelve una lista vacía	
Then	el sistema retorna snapshot_count: 0 y snapshots: [] sin generar error.	

US-06	Consultar historial de análisis	Usuario
Como usuario , quiero ver todos los análisis realizados previamente, para no repetir análisis de sitios ya consultados y reutilizar la información.		
Given	existen análisis almacenados en la base de datos	
When	el usuario hace GET /api/analyses/	
Then	recibe lista ordenada por fecha descendente con id, url, status, duration _ms y cantidad de tecnologías detectadas.	
Given	no se han realizado análisis aun	
When	el usuario consulta GET /api/analyses/	
Then	recibe lista vacía [] con código 200.	

US-07	Ver detalle completo de un análisis	Usuario
Como usuario , quiero consultar el detalle completo de un análisis específico, incluyendo la evidencia de detección de cada tecnología.		
Given	existe un análisis con id=1	
When	el usuario hace GET /api/analyses/1/	
Then	recibe el análisis completo con todas las tecnologías y su campo evidence explicando que patrón o header originó cada detección.	
Given	el id solicitado no existe en la base de datos	
When	el usuario consulta GET /api/analyses/999/	
Then	el sistema retorna 404 con mensaje {"error": "Análisis no encontrado"}.	

US-08	Comparar las tecnologías de dos sitios	Usuario
Como usuario , quiero comparar las tecnologías de dos sitios analizados, para identificar similitudes y diferencias en sus stacks tecnológicos.		
Given	existen dos analisis con id=1 e id=2	
When	el usuario hace GET /api/compare/?a=1&b;=2	
Then	recibe JSON con tres listas: common (tecnologías en común), only_a (exclusivas del sitio A) y only_b (exclusivas del sitio B).	
Given	alguno de los ids enviados no existe	
When	el usuario consulta el endpoint con un id invalido	
Then	el sistema retorna 404 indicando cual de los dos analisis no fue encontrado.	

US-09	Consultar estadísticas globales	Usuario
Como usuario , quiero ver cuales son las tecnologías más detectadas en todos los análisis, para identificar tendencias del ecosistema web.		
Given	existen múltiples análisis almacenados	
When	el usuario hace GET /api/stats/	
Then	recibe total_analyses, total_techs y top_technologies: ranking de las 20 más frecuentes con nombre, categoría y conteo.	

US-10	Orquestar todos los servicios desde un punto único (Facade)	Sistema
Como sistema , debo orquestar FrontendDetector, BackendDetector, CDNDetector, DNSService, WhoisService, GeoService y WaybackService desde el metodo analyze(), compartiendo la petición HTTP entre los detectores que la necesiten.		
Given	el Analyzer recibe una URL para analizar	
When	ejecuta el método analyze() aplicando el Facade Pattern	
Then	realiza exactamente una petición HTTP al sitio, la comparte entre FrontendDetector, BackendDetector y CDNDetector, y llama independientemente a DNSService, WhoisService, GeoService y WaybackService, consolidando todo en un JSON.	
Given	uno de los servicios externos falla durante el análisis	
When	se produce una excepcion en WhoisService, GeoService o WaybackService	
Then	el sistema registra el error en logs, retorna null en ese campo específico y completa el resto del análisis con normalidad.	

US-11	Detectar tecnologías frontend sin falsos positivos	Sistema
Como sistema , debo detectar frameworks de frontend analizando HTML, scripts y headers, aplicando un sistema de puntaje para evitar falsos positivos.		
Given	el HTML de un sitio contiene id="root" y un script con react.production.min.js	
When	el FrontendDetector evalúa las firmas de React	
Then	detecta React con confidence >= 60 y registra la evidencia específica que originó la detección.	
Given	el HTML contiene solo un patrón débil de Angular como main.js	
When	el FrontendDetector evalúa las firmas de Angular	
Then	no reporta Angular porque requiere al menos 2 señales fuertes para evitar falsos positivos.	

US-12	Interactuar mediante interfaz web centralizada	Usuario
Como usuario , quiero disponer de un formulario web intuitivo para ingresar una URL y ver los resultados tabulados, para evitar tener que consumir de forma manual o programática los endpoints de la API REST.		
Given	El usuario se encuentra en la pantalla principal de la aplicación web de Spynet y digita una URL válida en el componente de entrada principal.	
When	Presiona el botón de ejecución o acciona el envío del formulario.	
Then	El sistema procesa la solicitud mediante el orquestador (<i>Facade</i>) y distribuye visualmente las respuestas en contenedores específicos para tecnologías, DNS, WHOIS, localización y snapshots históricos.	

US-13	Visualizar geolocalización en mapa interactivo	Usuario
Como usuario , quiero que las coordenadas geográficas de la IP resuelta se proyecten sobre un mapa digital interactivo, para comprender de forma especial e inmediata dónde reside físicamente la infraestructura del sitio analizado.		
Given	El GeoService retorna exitosamente un objeto con los campos latitud y longitud válidos provistos por la API pública.	
When	El frontend carga el bloque de geolocalización.	
Then	Renderiza de manera interactiva un mapa digital centrando la cámara y colocando un PIN o marcador en las coordenadas exactas, complementando visualmente los campos de texto de país, ciudad e ISP.	
Given	El servicio de geolocalización externo no se encuentra disponible o retorna coordenadas nulas (null).	
When	El frontend procesa la respuesta del JSON general.	
Then	Oculta el componente del mapa interactivo o despliega en su lugar un mensaje controlado: <i>"Ubicación en mapa no disponible para este análisis"</i> , manteniendo intactas las demás tarjetas funcionales.	

US-14	Inspeccionar evolución visual mediante snapshots	Usuario
Como usuario , quiero visualizar las capturas históricas en una línea de tiempo ordenada con enlaces de navegación interactivos, para estudiar la evolución del diseño y las interfaces del dominio evaluado a lo largo del tiempo		
Given	El análisis del backend expone un arreglo indexado con los timestamps e imágenes/URLs de archive.org.	
When	Se despliega el módulo inferior de históricos en el frontend	
Then	Estructura una interfaz de línea de tiempo con miniaturas clicables que, al ser accionadas por el usuario, abren de forma segura (target="_blank") la versión exacta archivada en los servidores de Internet Archive.	
Given	La API CDX retorna un listado vacío debido a la inexistencia de registros históricos para el dominio consultado.	
When	Se mapea el bloque de datos del historial.	
Then	Imprime en pantalla un contenedor limpio que indica <i>"Sin registros históricos disponibles"</i> , con un estado interno de snapshot_count: 0 sin levantar excepciones de ejecución.	

US-15	Visualizar estadísticas globales mediante componentes gráficos	Usuario
Como usuario , quiero observar el ranking de tecnologías analizadas mediante gráficas dinámicas de barras o pastel, para identificar de forma estadística y visual cuáles son los stacks más populares en los sitios web auditados.		
Given	El usuario ingresa a la sección global de analíticas (/stats)	
When	El backend despacha el arreglo JSON con el consolidado del top 20 de tecnologías recopiladas en la base de datos.	
Then	El frontend inyecta los sets de datos en componentes visuales (como Chart.js o D3) renderizando gráficas interactivas organizadas por categorías (Frontend, Backend, CDNs).	

US-16	Análisis tecnológico de versiones históricas (Soporta RF-19)	Usuario
Como usuario , quiero ordenar el escaneo tecnológico enfocado en un snapshot específico del pasado, para trazar un paralelo técnico comparativo entre las tecnologías que usaba el sitio web antes y las que usa actualmente.		
Given	El usuario selecciona un snapshot histórico específico en la interfaz	
When	Solicita ejecutar el escaneo retroactivo de esa captura.	
Then	El sistema descarga el HTML crudo guardado, procesa firmas pasivas de frontend/backend, almacena el resultado y no dispara consultas sincrónicas de red (DNS, WHOIS o GeoIP)	
Given	La API CDX retorna un listado vacío debido a la inexistencia de registros históricos para el dominio consultado.	
When	Se mapea el bloque de datos del historial.	
Then	Imprime en pantalla un contenedor limpio que indica <i>"Sin registros históricos disponibles"</i> , con un estado interno de snapshot_count: 0 sin levantar excepciones de ejecución.	

US-17	Obtener un reporte o diagnóstico interpretativo por IA	Usuario
Como usuario , quiero que un modelo de inteligencia artificial analice textualmente los resultados técnicos agregados del sitio, para obtener un resumen ejecutivo comprensible que evalúe el estado general, stack y seguridad de la infraestructura sin tener que interpretar manualmente los logs técnicos.		
Given	El orquestador del sistema ha culminado exitosamente la recolección de datos de DNS, WHOIS, Geolocalización y Tecnologías.	
When	El backend envía un prompt estructurado con estos datos crudos adjuntos hacia el endpoint de la API de IA gratuita (AIService).	
Then	El sistema procesa la respuesta e imprime en la interfaz web un párrafo de texto legible que actúa como "Diagnóstico Asistido por IA" o "Resumen de Auditoría".	
Given	El backend ejecuta el llamado sincrónico/asincrónico hacia el proveedor de IA.	
When	Intercepta la falla de manera silenciosa devolviendo un campo ai_analysis: null, y el frontend muestra un aviso amigable como <i>"Análisis inteligente temporalmente saturado"</i> sin detener ni afectar la carga de los DNS o el resto de los componentes.	

3. User Story Mapping

El mapa organiza las historias de usuario en tres niveles: **Actividades** (objetivos de alto nivel del usuario segun la propuesta), **Tareas** (acciones concretas que las componen) e **Historias por sprint** (priorizacion iterativa del desarrollo). Se planifican tres sprints alineados con las funcionalidades de la propuesta.

Actividades	Analizar sitios web			Dominio e infraestructura		Historial y monitoreo		Estadísticas y métricas	
Tareas	Analisis	Interfaz de usuario	Analisis avanzado	Consultas de red y propiedad	Geolocalizacion	Gestion de analisis	Archivamiento	Comparativas	Analitica de datos
Sprint 1	US-01 US-10 US-11			US-02 US-03		US-06			
Sprint 2		US-12			US-04 US-13	US-07	US-05 US-14	US-08	US-09 US-15
Sprint 3			US-17				US-16		

3.1 Priorizacion MoSCoW

Las historias se clasifican según el método MoSCoW: **Must have** (imprescindible para la entrega), **Should have** (importante pero no crítico), **Could have** (deseable si hay tiempo) y **Won't have** (fuera del alcance actual).

ID	Historia	Prioridad	Sprint
US-01	Analizar un sitio web (tecnologias)	Must have	Sprint 1
US-02	Consultar registros DNS del dominio	Must have	Sprint 1
US-03	Consultar datos WHOIS del dominio	Must have	Sprint 1
US-04	Ver geolocalización de la IP	Should have	Sprint 2
US-05	Ver historial Wayback Machine	Should have	Sprint 2
US-06	Consultar historial de análisis	Must have	Sprint 1
US-07	Ver detalle completo de un análisis	Must have	Sprint 2
US-08	Comparar tecnologias de dos sitios	Should have	Sprint 2
US-09	Consultar estadísticas globales	Should have	Sprint 2
US-10	Facade: orquestar todos los servicios	Must have	Sprint 1
US-11	Detección frontend sin falsos positivos	Must have	Sprint 1
US-12	Interfaz web de análisis centralizada	Must have	Sprint 2
US-13	Proyección de localización en mapa dinámico	Should have	Sprint 2

US-14	Línea de tiempo interactiva de snapshots	Should have	Sprint 2
US-15	Graficación visual de analíticas globales	Should have	Sprint 2
US-16	Análisis tecnológico retroactivo de código histórico	Could have	Sprint 3
US-17	Reporte interpretativo asistido por IA	Could have	Sprint 3

4. Referencias

- [1] Django Project. (2024). *Django documentation v4.2*. <https://docs.djangoproject.com/en/4.2/>
- [2] Django REST Framework. (2024). *DRF documentation*. <https://www.django-rest-framework.org/>
- [3] Internet Archive. (2024). *Wayback Machine CDX API*. https://archive.org/help/wayback_api.php
- [4] ip-api.com. (2024). *IP Geolocation API documentation*. <https://ip-api.com/docs>
- [5] Patton, J. (2014). *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media.
- [6] Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.
- [7] plan.io. (2024). *User Story Mapping: A Complete Guide*. <https://plan.io/blog/user-story-mapping/>
- [8] Wappalyzer. (2024). *Open source technology profiler*. <https://github.com/wappalyzer/wappalyzer>
- [9] Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.